

Attention Is All You Need 논문 요약

쿠빅 23기 조호평

쿠빅 23기 조호평

0. Abstract

1. Introduction

1.1 기존 접근의 한계 — RNN/LSTM 계열

1.2 Attention 메커니즘의 등장

1.3 Transformer의 등장

2. Background

2.1 Sequential computation 줄이기

2.2 Self-attention

3. Model Architecture

전체 구조인 Encoder-Decoder

3.1 Encoder and Decoder Stacks

3.2 Attention

3.2.1 Scaled Dot-Product Attention

3.2.2 Multi-Head Attention

3.3 Position-wise Feed-Forward Networks

3.4 Embeddings and Softmax

3.5 Positional Encoding

4. Why Self-Attention

Table 1 — 네 가지 layer type 비교

5. Training

5.1 Training Data and Batching

5.2 Hardware and Schedule

5.3 Optimizer

5.4 Regularization

6. Results

6.2 Model Variations (Table 3 — Ablation, EN-DE dev set 기준)

7. Conclusion

저자 : Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

0. Abstract

Sequence transduction) 모델은 인코더와 디코더를 포함하는 복잡한 순환 신경망 또는 합성곱 신경망에 기반한다. 최고 성능 모델들은 인코더와 디코더를 attention 메커니즘으로 연결한다. 이 논문은 **순환(recurrence)과 합성곱(convolution)을 완전히 제외하고, attention 메커니즘에만 전적으로 기반한 새로운 아키텍처 Transformer**를 제안한다.

1. Introduction

1.1 기존 접근의 한계 — RNN/LSTM 계열

RNN, 특히 LSTM과 gated RNN은 언어 모델링과 기계번역 같은 시퀀스 모델링/변환 문제에서 SOTA로 확고히 자리잡아 왔다.

근본적 제약은? ⇒ 순환 모델은 일반적으로 입출력 시퀀스의 심볼 위치에 따라 계산을 나눈다. 위치를 계산 시간의 스텝에 정렬시켜, 이전 은닉 상태 h_{t-1} 과 위치 t 의 입력의 함수로 은닉 상태 h_t 의 시퀀스를 생성한다. 이 **순차적 특성** 때문에 학습 샘플 내에서의 병렬화가 불가능해지며, 시퀀스 길이가 길어질수록 메모리 제약으로 배치 처리가 제한되어 이 문제가 더 치명적으로 작용한다. 인수분해 트릭이나 조건부 계산 등으로 효율성이 개선되긴 했지만, 순차 계산이라는 근본적 제약 자체는 여전히 남아있다.

1.2 Attention 메커니즘의 등장

Attention 메커니즘은 입력이나 출력 시퀀스 내 거리에 무관하게 의존성을 모델링할 수 있게 해주면서, 다양한 과제에서 시퀀스 모델링/변환 모델의 핵심 요소가 되었다. 그러나 극소수 경우를 제외하면, 이런 attention 메커니즘은 대부분 순환 신경망과 결합되어 사용된다. 즉 여전히 RNN이 중심이고 attention은 보조 장치였다.

1.3 Transformer의 등장

이 논문에서는 순환을 배제하고 대신 입력과 출력 사이의 전역 의존성을 이끌어내기 위해 전적으로 attention 메커니즘에만 의존하는 모델 Transformer를 제안한다. 훨씬 더 많은 병렬화를 가능하게 하며, 8개의 P100 GPU로 단 12시간만 학습해도 새로운 SOTA 번역 품질에 도달할 수 있다.

2. Background

2.1 Sequential computation 줄이기

Extended Neural GPU, ByteNet, ConvS2S는 모두 CNN을 기본 빌딩 블록으로 써서, 이 목표(순차 계산 줄이기)를 추구해온 모델들이다. 이 모델들은 모든 입출력 position에 대해 hidden representation을 병렬로 계산한다.

한계: 이런 CNN 기반 모델에서, 서로 다른 두 position의 신호를 연관짓는 데 필요한 연산 수는 position 간 거리에 비례해서 늘어난다. ConvS2S는 선형으로, ByteNet은 로그 스케일로. 이 때문에 멀리 떨어진 position 간의 dependency를 학습하기가 더 어려워진다.

Transformer의 개선: 이걸 **constant 횟수의 연산**으로 줄였다. 다만 attention-weighted position을 평균내는 과정에서 effective resolution이 떨어지는 대가가 있는데, 이건 Multi-Head Attention으로 상쇄한다.

2.2 Self-attention

Self-attention(intra-attention이라고도 함)은 하나의 시퀀스 내 서로 다른 position들을 관계지어서, 그 시퀀스의 representation을 계산하는 attention 메커니즘이다. Reading comprehension, abstractive summarization, textual entailment, task-independent sentence representation 학습 등 다양한 task에서 이미 성공적으로 쓰여 왔다.

3. Model Architecture

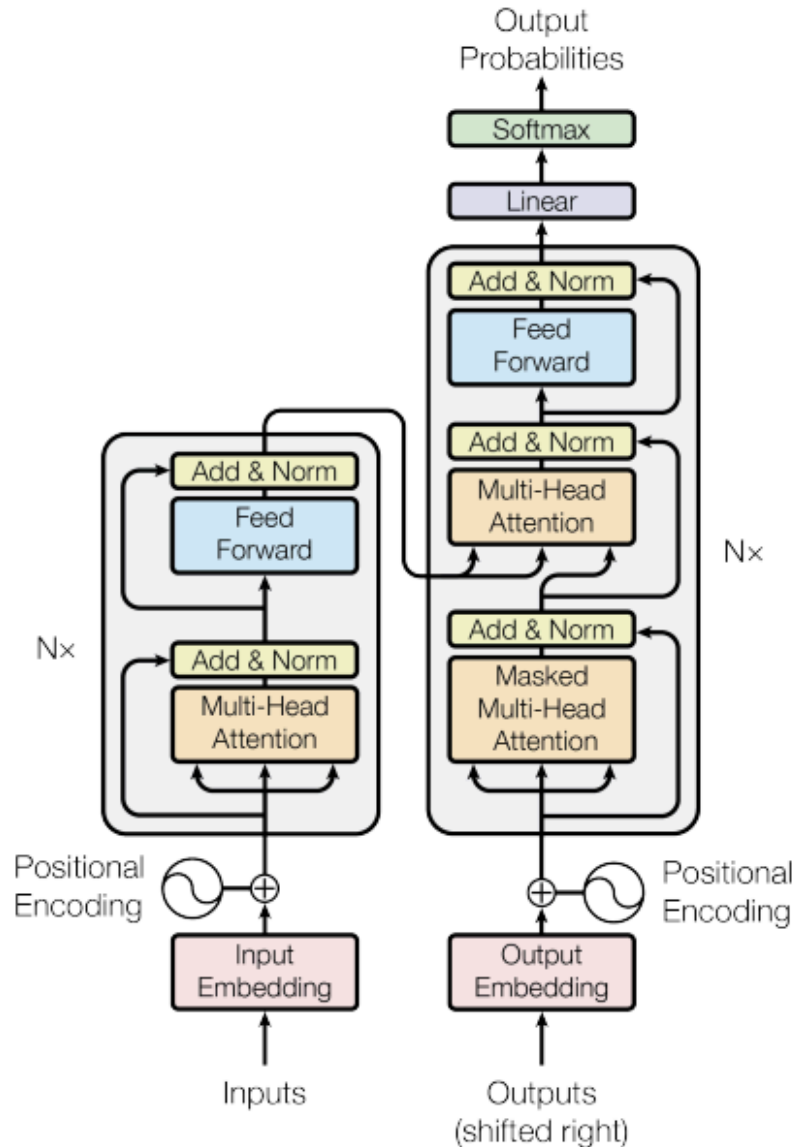


Figure 1: The Transformer - model architecture.

전체 구조인 Encoder-Decoder

대부분의 competitive neural sequence transduction model은 encoder-decoder 구조를 갖는다.

- **Encoder:** 입력 symbol representation 시퀀스 $(x_1, \dots, x_n)$ 을 continuous representation 시퀀스 $z = (z_1, \dots, z_n)$ 으로 매핑
- **Decoder:** z 가 주어지면, output 시퀀스 $(y_1, \dots, y_m)$ 을 한 번에 한 element씩 생성
- 각 스텝에서 모델은 **auto-regressive**하다. 이전에 생성한 symbol들을 다음 생성 시 추가 입력으로 사용

Transformer는 이 구조를 그대로 따르되, encoder와 decoder 양쪽 모두에서 **stacked self-attention + point-wise fully connected layer**를 쌓는 방식을 쓴다 (Figure 1).

3.1 Encoder and Decoder Stacks

Encoder

- $N = 6$ 개의 동일한 layer를 쌓은 stack
- 각 layer는 두 개의 sub-layer로 구성:
 1. multi-head self-attention
 2. simple, position-wise fully connected feed-forward network
- 각 sub-layer 주변에 **residual connection**을 두고, 그 뒤에 **layer normalization** 적용:

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- residual connection을 위해, embedding layer를 포함한 모델의 모든 sub-layer는 output dimension $d_{\text{model}} = 512$ 로 통일

Decoder

- 마찬가지로 $N = 6$ 개의 동일한 layer stack
- encoder의 두 sub-layer에 더해, **세 번째 sub-layer**를 추가한다. Encoder stack의 output에 대해 multi-head attention을 수행
- encoder와 마찬가지로 각 sub-layer에 residual connection + layer normalization 적용
- decoder의 self-attention sub-layer는 **뒤쪽 position을 못 보게 수정된다**. 이 masking과, output embedding이 한 position만큼 offset되어 있다는 사실이 합쳐져서, position i 의 예측이 오직 i 보다 이전 position의 known output에만 의존하도록 보장한다.

3.2 Attention

Attention function은 query와 key-value 쌍의 집합을 output에 매핑하는 것으로 설명할 수 있다. Query, key, value, output은 모두 벡터다. Output은 value들의 weighted sum으로 계산되며, 각 value에 할당되는 weight는 해당 key와 query 간의 compatibility function으로 계산된다.

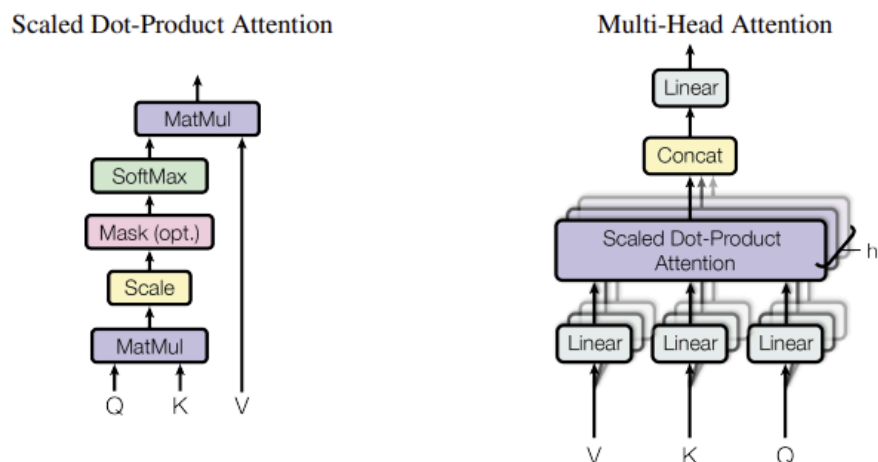


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

3.2.1 Scaled Dot-Product Attention

Input: dimension d_k 의 query와 key, dimension d_v 의 value. Query와 모든 key의 dot product를 계산하고, 각각을 $\sqrt{d_k}$ 로 나눈 뒤 softmax를 적용해서 value에 대한 weight를 얻는다.

실제로는 query들을 묶은 행렬 Q 에 대해 한 번에 계산하며, key와 value도 각각 K , V 행렬로 묶는다.

왜 $\text{scale}(\sqrt{d_k})$ 로 나누기)이 필요한가: additive attention과 dot-product attention 두 가지가 가장 흔히 쓰이는데, dot-product attention은 이 scaling factor만 빼면 이 논문 알고리즘과 동일하다. Dot-product attention이 속도·공간 효율 면에서 훨씬 낮지만(최적화된 행렬곱 코드 활용 가능), d_k 가 작을 땐 두 방식 성능이 비슷한 반면 d_k 가 클수록 **scaling 없는 dot-product attention은 additive attention보다 성능이 떨어진다**. 이는 d_k 가 커지면 dot product 값 자체가 커져서, softmax를 **gradient가 극히 작은 영역**으로 밀어넣기 때문이라고 추정된다.

3.2.2 Multi-Head Attention

d_{model} 차원의 query/key/value로 단일 attention function을 수행하는 대신, query/key/value를 각각 d_k , d_k , d_v 차원으로 **h 번 서로 다르게 학습된 linear projection**을 거쳐 projection한 뒤, 이 projection된 버전들에 대해 병렬로 attention function을 수행하는 것이 더 유리하다는 것을 발견했다. 각 projection이 d_v 차원 output을 내고, 이들을 concatenate한 뒤 다시 한 번 projection해서 최종 값을 얻는다 (Figure 2).

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Projection들은 파라미터 행렬이다:

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, \quad W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

왜 여러 head가 필요한가: Multi-head attention은 모델이 서로 다른 position에서, 서로 다른 representation subspace의 정보에 **동시에 attend**할 수 있게 해준다. Single attention head 하나만 쓰면 averaging이 이걸 억제해버린다.

이 논문에서는 $h = 8$ 개의 병렬 attention layer(head)를 사용했고, 각각에 대해 $d_k = d_v = d_{\text{model}}/h = 64$ 를 썼다. 각 head의 차원이 줄어든 덕분에, 전체 계산 비용은 full dimensionality의 single-head attention과 비슷한 수준이다.

3.3 Position-wise Feed-Forward Networks

Attention sub-layer에 더해, encoder와 decoder의 각 layer는 fully connected feed-forward network를 포함한다. 이건 각 position에 개별적으로, 동일하게 적용된다. ReLU를 사이에 둔 두 개의 linear transformation으로 구성된다.

Linear transformation 자체는 position마다 동일하지만, layer마다 서로 다른 파라미터를 쓴다. 다르게 표현하면 kernel size 1인 두 개의 convolution이라고도 볼 수 있다.

Input/output dimension은 $d_{\text{model}} = 512$, inner-layer dimension은 $d_{ff} = 2048$ 이다.

3.4 Embeddings and Softmax

다른 sequence transduction model들처럼, 학습된 embedding을 써서 input token과 output token을 d_{model} 차원 벡터로 변환한다. Decoder output을 다음 토큰 확률로 변환하는 데는 학습된 linear transformation과 softmax function을 쓴다. 이 모델에서는 **두 embedding layer와 pre-softmax linear transformation이 동일한 weight matrix를 공유**한다. Embedding layer에서는 이 weight에 $\sqrt{d_{\text{model}}}$ 을 곱한다.

3.5 Positional Encoding

모델에 recurrence도 convolution도 없기 때문에, 시퀀스의 순서 정보를 모델이 활용하려면 token의 relative/absolute position 정보를 별도로 주입해야 한다. 그래서 encoder와 decoder stack 맨 아래에서 input embedding에 "positional encoding"을 더한다. Positional encoding은 embedding과 같은 d_{model} 차원이라, 둘을 그냥 더할 수 있다.

이 논문에서는 서로 다른 frequency의 sine과 cosine 함수를 사용했다.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

4. Why Self-Attention

이 섹션은 self-attention layer를, 가변 길이 시퀀스 $(x_1, \dots, x_n)$ 을 같은 길이의 다른 시퀀스 $(z_1, \dots, z_n)$ 로 매핑하는 데 흔히 쓰이는 recurrent layer 및 convolutional layer와 비교한다 ($x_i, z_i \in \mathbb{R}^d$, 전형적인 encoder/decoder의 hidden layer가 이런 매핑을 한다).

Self-attention을 채택한 근거로 세 가지 desiderata를 고려한다:

1. **Layer당 총 computational complexity**
2. **병렬화 가능한 계산량** — 필요한 최소 sequential operation 수로 측정
3. **long-range dependency 간의 path length** — 네트워크 내에서 입출력 position 사이를 forward/backward 신호가 이동해야 하는 경로 길이. 이 경로가 짧을수록 long-range dependency를 학습하기 쉬워진다. 그래서 서로 다른 layer type에서 입출력 position 간 maximum path length도 함께 비교한다.

Table 1 — 네 가지 layer type 비교

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Self-attention: 상수 개수의 순차 실행 연산만으로 모든 position을 연결한다 ($O(1)$ sequential operations, $O(1)$ path length). 반면 recurrent layer는 $O(n)$ 번의 순차 연산이 필요하다.

Computational complexity 비교: 시퀀스 길이 n 이 representation dimension d 보다 작을 때는 word-piece나 byte-pair representation을 쓰는 최신 기계번역 모델의 문장 representation에서 흔한 경우다. self-attention layer가 recurrent layer보다 빠르다.

매우 긴 시퀀스를 다룰 때의 개선책: computational performance를 높이기 위해, self-attention을 입력 시퀀스에서 해당 output position을 중심으로 한 크기 r 의 neighborhood만 고려하도록 제한할 수 있다. 이러면 maximum path length가 $O(n/r)$ 로 늘어난다.

Convolutional layer: kernel width $k < n$ 인 단일 convolutional layer는 모든 입출력 position 쌍을 연결하지 못한다. 모든 쌍을 연결하려면 contiguous kernel의 경우 $O(n/k)$ 개의 conv layer stack이, dilated convolution의 경우 $O(\log_k(n))$ 개가 필요해서, 네트워크 내 가장 긴 경로 길이가 늘어난다. Convolutional layer는 일반적으로 recurrent layer보다 k 배만큼 더 비싸다. Separable convolution은 complexity를 $O(k \cdot n \cdot d + n \cdot d^2)$ 까지 상당히 낮춰주지만, $k = n$ 인 경우에도 separable convolution의 complexity는 이 논문이 채택한 접근(self-attention + point-wise feed-forward layer 조합)과 동일한 수준이다.

5. Training

5.1 Training Data and Batching

- **EN-DE:** 표준 WMT 2014 English-German 데이터셋, 약 450만 문장쌍. Byte-pair encoding으로 인코딩, source-target 공유 vocabulary 약 37,000 토큰.
- **EN-FR:** 훨씬 큰 WMT 2014 English-French 데이터셋, 3,600만 문장. Word-piece vocabulary 32,000 토큰으로 분리.
- 문장쌍은 대략적인 시퀀스 길이 기준으로 배치 구성. 각 학습 배치는 약 25,000개 source token + 25,000개 target token을 포함.

5.2 Hardware and Schedule

- 8개 NVIDIA P100 GPU 1대에서 학습.
- **Base model:** step당 약 0.4초, 총 100,000 step (12시간) 학습.
- **Big model:** step당 약 1.0초, 총 300,000 step (3.5일) 학습.

5.3 Optimizer

Adam optimizer 사용, $\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$. Learning rate는 학습 과정에서 다음 공식에 따라 변화:

즉 처음 $warmup_steps$ 동안은 learning rate를 선형으로 증가시키고, 그 이후엔 step number의 inverse square root에 비례해서 감소시킨다. $warmup_steps = 4000$ 사용.

5.4 Regularization

세 가지 regularization 기법 적용:

- **Residual Dropout:** 각 sub-layer output에, sub-layer input에 더해지고 정규화되기 전 단계에서 dropout 적용. embedding과 positional encoding의 합에도 encoder-decoder 양쪽에서 dropout 적용. Base model은 $P_{drop} = 0.1$.
- **Label Smoothing:** $\epsilon_{ls} = 0.1$ 의 label smoothing 적용. 이는 perplexity를 악화시키는데(모델이 더 불확실해지도록 학습하므로), accuracy와 BLEU score는 개선시킨다.

6. Results

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Table 2 핵심: Transformer가 기존 모델 대비 BLEU는 대등하거나 더 높으면서, Training Cost(FLOPs)는 자릿수 단위로 훨씬 낮다. 예를 들어 EN-FR에서 ConvS2S Ensemble의 1.2×10^{21} FLOPs 대비 Transformer(big)는 2.3×10^{19} FLOPs이다.

6.2 Model Variations (Table 3 — Ablation, EN-DE dev set 기준)

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)		positional embedding instead of sinusoids								4.92	25.7	
big	6	1024	4096	16			0.3		300K	4.33	26.4	213

- **(A) head 수 및 d_k, d_v 변화** (계산량은 고정): single-head attention은 최적 설정 대비 0.9 BLEU 낮음. 반대로 head를 너무 많이 늘려도 품질이 떨어짐. **head 수에는 sweet spot이 있다.**
- **(B) attention key size d_k 축소:** 모델 품질이 나빠짐. dot product보다 더 정교한 compatibility function이 유리할 수 있음을 시사.
- **(C), (D) 모델 크기 및 dropout:** 예상대로 모델이 클수록 성능이 좋고, dropout은 overfitting 방지에 확실히 도움됨.
- **(E) Positional encoding:** sinusoidal 대신 learned positional embedding으로 교체해도 base model과 거의 동일한 결과. 두 방식 간 성능 차이는 미미함.

7. Conclusion

- Transformer는 recurrent layer를 multi-head self-attention으로 완전히 대체한, **attention에만 전적으로 기반한 최초의 sequence transduction model**이다.
- 번역 과제에서 recurrent-convolutional 기반 아키텍처보다 현저히 빠르게 학습 가능. EN-DE, EN-FR 양쪽에서 새 SOTA 달성, EN-DE에서는 기존 모든 published 앙상블도 능가.